

Data Model

Victor Eijkhout

2026 PCSE

Shared and private data

You have already seen some of the basics:

- Data declared outside a parallel region is shared.
- Data declared in the parallel region is private.
(Fortran does not have this block scope mechanism)

```
1 int i;  
2 #pragma omp parallel  
3 { double i; .... }
```

- You can change all this with clauses *private* / *shared*:

```
1 int i;  
2 #pragma omp parallel private(i)
```

Variables in loops

```
1 int i; double t;  
2 #pragma omp parallel for  
3   for (i=0; i<N; i++) {  
4     t = sin(i*pi*h);  
5     x[i] = t*t;  
6   }
```

- The loop variable is automatically private.
- The temporary `t` is shared, but conceptually private to each iteration: needs to be declared private.
(What happens if you don't?)

Copying to/from private data

- Private data is uninitialized

```
1 int i = 3;  
2 #pragma omp parallel private(i)  
3   printf("%d\n",i); // undefined!
```

- To import a value into a private variable:

```
1 int i = 3;  
2 #pragma omp parallel firstprivate(i)  
3   printf("%d\n",i); // prints 3
```

- `lastprivate` to preserve value of last iteration.

```
1 float x;  
2 #pragma omp parallel for lastprivate(x)  
3 for ( int i=0; i<1000; i++ ) {  
4   x = f(i); g(x);  
5 }  
6 // now x is f(999)
```

Default behaviour

- `default(shared)` OR `default(private)`
- useful for debugging: `default(none)`
because you have to specify everything as shared/private

Persistent thread data

- Private data disappears after the parallel region.
What if you want data to persist?

- Directive *threadprivate*

```
1 double seed;  
2 #pragma omp threadprivate(seed)
```

- Standard application: random number generation.
- Tricky: has to be global or static.

Arrays

- Statically allocated arrays can be made private.
- Dynamically allocated ones can not: the pointer becomes private.